

Xử lý Đọc/ghi file với Java

Phân Loại Đọc/Ghi File

- Khi đọc/ghi file trong Java, có 2 loại chính:
- **Character Streams** (Luồng ký tự): đã được giải mã thành ký tự Unicode, ví dụ file văn bản/ký tự: .txt, .csv, .java, .html...
- **Byte Streams** (Luồng nhị phân): xử lý dữ liệu dưới dạng byte thô, dùng cho mọi loại file (image, audio, video, file đã biên dịch .bin, .dat...)

Luồng Ký tự (Character Streams)

Class	Mục đích sử dụng
FileReader & FileWriter	Đọc/Ghi file text cơ bản theo từng ký tự
PrintWriter	Tương tự FileWriter, cung cấp theo các method để “format” data trước khi lưu
BufferedReader & BufferedWriter	Luôn dùng khi đọc/ghi file text thực tế vì: • Đọc theo dòng (readLine). • Buffer tăng tốc độ.

Ưu tiên:

- Đọc file với **BufferedReader**
- Ghi file với **BufferedWriter + PrintWriter**

Luồng nhị phân (Byte Streams)

Class	Mục đích sử dụng
FileInputStream & FileOutputStream	Đọc/Ghi file nhị phân cơ bản theo từng byte
BufferedInputStream & BufferedOutputStream	Sử dụng buffer, giúp tăng tốc độ đọc/ghi file nhị phân lớn

Đọc file với FileReader

1. FileReader là gì?

- **FileReader** là class dùng để đọc file văn bản (text file), **theo từng ký tự** (character).
- Phù hợp khi bạn cần đọc toàn bộ file, duyệt từng ký tự riêng lẻ.

▪ Cú pháp:

- **FileReader** fr = new **FileReader**("input.txt"); // Mở file

int c;

while ((c = fr.read()) != -1) { // Đọc từng ký tự, trả về int

System.out.print((char) c); // Ép int về char để hiển thị

}

- **fr.read()** đọc 1 ký tự, trả về mã ASCII hoặc UTF-16 (Unicode) dạng int.
- **Trả về -1** khi tới cuối file (EOF).

Ghi file với FileWriter

- **FileWriter** là class dùng để ghi dữ liệu text vào file, từng ký tự hoặc chuỗi.

- **Cú pháp:**

```
FileWriter fw = new FileWriter("output.txt");  
fw.write("Hello, world!¥n");  
fw.write("This is FileWriter example.¥n");
```

- **fw.write()** ghi dữ liệu vào file.
- File sẽ được tạo mới nếu chưa tồn tại, hoặc ghi đè nếu đã tồn tại.

PrintWriter – Ghi file văn bản dễ dàng hơn

- Hỗ trợ các phương thức quen thuộc như print(), println(), printf() giống System.out
- **Cú pháp:**

```
PrintWriter pw = new PrintWriter("output2.txt");  
pw.println("Hello, PrintWriter!");  
pw.printf("Number: %d, String: %s\n", 10, "Java");
```

Đọc/Ghi file với BufferedReader/BufferedWriter

- 1. Buffer là gì?

Buffer (bộ đệm) là vùng nhớ tạm nằm giữa chương trình và thiết bị IO (ổ cứng, mạng).

- **Khi đọc file:** thay vì đọc từng ký tự hoặc byte từ ổ cứng
→ chương trình đọc một khối lớn dữ liệu (chunk) vào buffer, sau đó lấy dần từng phần trong RAM → nhanh hơn.
- **Khi ghi file:** thay vì ghi từng ký tự xuống ổ cứng (rất tốn thời gian IO)
→ chương trình ghi dữ liệu vào buffer trước, khi buffer đầy hoặc đóng stream
→ ghi một lần xuống ổ cứng → tiết kiệm thời gian.

Đọc/Ghi file với BufferedReader/BufferedWriter

BufferedReader là gì?

- BufferedReader là class dùng để:
 - Đọc file text theo từng dòng (readLine()).
 - Đọc nhanh hơn FileReader nhờ buffer nội bộ

• **Cú pháp:**

```
FileReader fr = new FileReader("input.txt");  
BufferedReader br = new BufferedReader(fr);
```

```
String line;  
while ((line = br.readLine()) != null) { // đọc từng dòng  
    System.out.println(line);  
}
```

Đọc/Ghi file với **BufferedReader/BufferedWriter**

- **BufferedWriter** là class dùng để:
Ghi file text nhanh hơn **FileWriter** nhờ buffer.
Hỗ trợ **newLine()** để xuống dòng.

- **Cú pháp:**

```
FileWriter fw = new FileWriter("output.txt");  
BufferedWriter bw = new BufferedWriter(fw);
```

```
bw.write("Hello, BufferedWriter!");  
bw.newLine(); // xuống dòng  
bw.write("This is fast writing example.");
```

Sử dụng java.nio.file.Files (Character Stream)

Files là class utility trong package java.nio.file, cung cấp các phương thức static để thao tác file và thư mục như: đọc file, ghi file, copy, move, delete file.

- **Ưu điểm:**
- Code ngắn gọn, dễ dùng.
- Không cần tạo nhiều object như FileReader, BufferedReader.
- Hỗ trợ đọc ghi file text nhanh chóng cho **file nhỏ – vừa**.

Sử dụng java.nio.file.Files (Character Stream)

- Đọc file text bằng Files

```
List<String> lines = Files.readAllLines(Paths.get("input.txt"));  
for (String line : lines) {  
    System.out.println(line);  
}
```

- Ghi file text bằng Files

```
List<String> lines = Arrays.asList(  
    "Hello, java.nio.file.Files!",  
    "This is write() example."  
);  
Files.write(Paths.get("output.txt"), lines);
```

Bài Tập Đọc/Ghi File (Character Streams)

- Yêu cầu:
- Cho file “**students.txt**” chứa dữ liệu sinh viên, mỗi dòng có định dạng:
 - **Name,Score**
- Ví dụ định dạng của students.txt:
 - Bình,8.5
 - Nam,6.0
 - An,9.2
 - Tuấn,7.5

Viết chương trình

- Bước 1: Đọc file “students.txt” sử dụng **BufferedReader**.
- Bước 2: Với mỗi dòng, tách thành **tên sinh viên** và **điểm** (double), lưu vào danh sách object **Student**.
- Bước 3: Tính điểm trung bình của lớp.
- Bước 4: Xuất báo cáo vào file “**report.txt**” sử dụng **PrintWriter**, với format sau:

===== STUDENT REPORT =====

Name	Score	Pass/Fail
------	-------	-----------

Bình	8.50	PASS
Nam	6.00	PASS
An	9.20	PASS
Tuấn	7.50	PASS

Average Score: 7.80

- Quy định Pass/Fail:
Điểm $\geq 5.0 \rightarrow PASS$

Điểm $< 5.0 \rightarrow FAIL$

- Bước 5: In ra console dòng thông báo:
Report generated successfully to report.txt

Đọc/Ghi file với FileInputStream & FileOutputStream

- FileInputStream

Ví dụ: Đọc file ảnh (input.jpg) và in số byte

- Download file hình ảnh trong video [tại đây](#)

```
FileInputStream fis = new FileInputStream(filePath)
```

```
int sizeInBytes = fis.available(); // available trả về số byte có thể đọc
```

```
double sizeInMB = (double) sizeInBytes / (1024 * 1024);
```

```
System.out.printf("Kích thước file: %.2f MB\n", sizeInMB);
```

Đọc/Ghi file với FileInputStream & FileOutputStream

- FileOutputStream

Ví dụ: copy file

```
String inputPath = "input.jpg";  
String outputPath = "copy.jpg";
```

```
FileInputStream fis = new FileInputStream(inputPath);  
FileOutputStream fos = new FileOutputStream(outputPath))
```

```
int b;  
while ((b = fis.read()) != -1) {  
    fos.write(b);  
}
```

Sử dụng java.nio.file.Files (Byte Stream)

- Files.readAllBytes()

```
byte[ ] data = Files.readAllBytes(Paths.get(filePath));
```

```
System.out.println("Đã đọc file: " + filePath);
```

```
System.out.println("Kích thước file: " + data.length + " bytes");
```

```
System.out.println("Giá trị byte đầu tiên: " + data[0]);
```

- Files.write()

```
byte[ ] data = {10, 20, 30, 40, 50}; // mảng byte mẫu
```

```
Files.write(Paths.get(filePath), data);
```

```
System.out.println("Đã ghi file: " + filePath);
```

Bài Tập Đọc/Ghi File (Byte Stream)

Yêu cầu: Viết chương trình Java thực hiện copy file

Bước 1: tạo đường dẫn file nguồn và file đích. Download file mẫu [tại đây](#)

String **sourcePath** = "demo.mp3"; // đường dẫn file gốc

String **destPath** = "copy.mp3"; // đường dẫn file copy

Bước 2: Mở file nguồn để đọc

Dùng **FileInputStream** để mở file nguồn.

Bọc **FileInputStream** trong **BufferedInputStream** để tăng tốc độ đọc.

Bước 3: Mở file đích để ghi

Dùng **FileOutputStream** để mở file đích.

Bọc **FileOutputStream** trong **BufferedOutputStream** để tăng tốc độ ghi.

Bước 4: Copy dữ liệu từ file nguồn sang file đích

Đọc từng byte từ **BufferedInputStream**.

Ghi từng byte đó vào **BufferedOutputStream**.

Dùng vòng lặp:

int b;

```
while ((b = bis.read()) != -1) {
```

```
    bos.write(b);
```

```
}
```

Bước 5: Đo thời gian chạy chương trình

Trước khi bắt đầu copy → lưu thời gian bắt đầu

Sau khi copy xong → lưu thời gian kết thúc

sử dụng: **System.currentTimeMillis();**

In ra kết quả: Copy completed in XXX ms

Serialization & Serializable Interface (Extra)

- Khái niệm Serialization là gì?
- **Serialization** là quá trình chuyển **object** thành **byte stream** (dãy byte).
- **Deserialization** là quá trình ngược lại – chuyển byte stream về object gốc.
- **Serializable không bắt buộc** để viết code Java bình thường:
 - Viết class, method, xử lý logic → không liên quan gì tới Serializable.
 - Nếu chỉ chạy chương trình, tính toán, xử lý dữ liệu trong RAM → không cần.
 - Khi nào sử dụng Serializable?
 - Khi bạn cần lưu object vào file hoặc database
 - Khi cần gửi object qua network
 - Khi sử dụng các framework Java (Spring Session / Redis cache...)

Serializable Interface là gì?

- Serializable là một marker interface trong Java:
public interface Serializable { }
(Marker interface: không có method)
- **Tác dụng:** Nếu class implements Serializable → JVM có thể:
- Biến object thành byte stream (serialize).
- Khôi phục object từ byte stream (deserialize).